

01

Vector

Has Both Magnitude and Direction

1.1 What Is a Vector?



Core skills you will master in this section:

- ▶ Definition and essence of vectors: having both magnitude and direction, as distinct from scalars.
- ▶ Geometric properties of vectors: understanding magnitude and direction through arrow diagrams.
- ▶ Distinguishing row vectors from column vectors: understanding their differences in arrangement, notation, and application.
- ▶ Transposing vectors: converting a row vector to a column vector, and vice versa.
- ▶ Creating and manipulating vectors with NumPy: including indexing and slicing of one-dimensional and two-dimensional arrays, as well as the transpose operation.
- ▶ The relationship between vectors and matrices: understanding where vectors fit within matrix structures.

A vector has both magnitude and direction, and is widely used in fields such as physics and computer science. This section first introduces the basic concept of a vector and uses examples such as velocity and displacement to illustrate the difference between vectors and scalars. It then introduces the concept of a vector field, showing how phenomena such as wind speed and water flow can be described using vector fields.

Next, it explains the magnitude and direction of a vector, emphasizing these properties through arrow diagrams. The section then introduces the RGB color model, explaining how color can be represented with a three-dimensional vector, and discusses the concept of vector dimension. It goes on to discuss the difference between row vectors and column vectors, analyzing their arrangement from the perspective of matrices and showing how transposition converts one into the other.

Finally, the section explains how to create one-dimensional and two-dimensional arrays with NumPy, demonstrates indexing and slicing operations, and uses Python code to show how row vectors and column vectors can be transposed into each other.

Vectors Are Everywhere

A vector is a quantity that has both magnitude and direction.

Geometrically, a vector is a directed line segment.

In contrast, a scalar is simply a numerical value that has magnitude but no direction.

The speed shown on a car's dashboard, say 60 kilometers per hour, is a scalar, because it represents only the magnitude of the speed.

However, if we add the direction of travel — for example, “traveling due east at 60 kilometers per hour” — this speed becomes a vector, because it now includes both a magnitude (60 kilometers per hour) and a specific direction (due east).

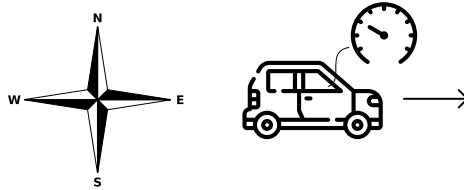


Figure 1. Speed, Scalar, and Vector

As another example, suppose that while hiking cross-country you are still 5 kilometers from your destination. This distance is a scalar, because it represents only magnitude and involves no direction.

But if we add a direction — for example, “5 kilometers to the southwest” — this quantity becomes a displacement, which is a vector. Displacement tells us not only how far to travel, but also in which direction.

Vectors are everywhere in everyday life. Phenomena such as wind, water flow, gravitational fields, and electromagnetic fields can all be described using vector fields. A vector field is a mathematical description in which every point in space is associated with a vector that represents the magnitude and direction of some physical quantity at that point.

For example, at any specific location, the wind has not only a speed (say, 10 meters per second) but also a definite direction (say, blowing from north to south).

The same is true of the water flow at a point in a river: it has both a flow speed and a flow direction.

The physical phenomenon at any single location can be described by a single vector. But when we look at an entire region, we find that wind speed and flow speed differ from place to place, together forming a vector field.

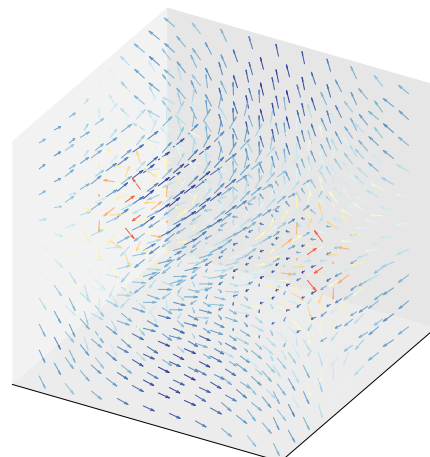
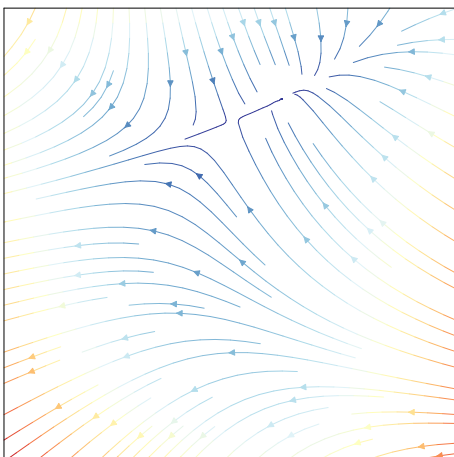


Figure 2. Vector Fields in the Plane and in Space

A Vector Has Both Magnitude and Direction

We can picture a vector as an arrow, which has two important properties: magnitude, the length of the arrow, and direction, the way the arrow points.

This is also why, in physics, a vector (向量) is often referred to in Chinese as 矢量, literally an “arrow quantity.”

As shown in Figure 3, the length of the arrow represents the magnitude of a vector, also called the length of a vector.

Simply put, the length of a vector is defined as the straight-line distance between its initial point (also called the start point, tail, or origin) and its terminal point (also called the end point or head).

The direction the arrowhead points represents the direction of the vector.

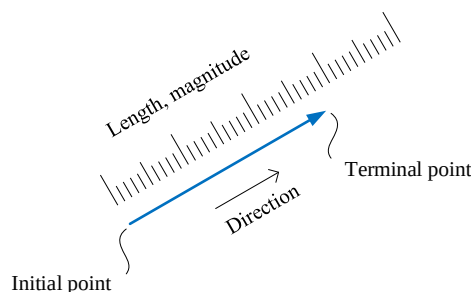


Figure 3. A Vector Has Both Magnitude and Direction

The magnitude of a vector is itself a scalar (it has magnitude but no direction).

As shown in Figure 4, even if two vectors have exactly the same magnitude, they represent different physical quantities as long as their directions differ. For example, one vector might point west while another points east; although they have the same length, they play completely different roles when describing phenomena such as force or velocity.

Figure 4: some vectors are perpendicular to each other, also called orthogonal; this means they are independent in their own directions and do not affect one another. Other vectors point in exactly opposite directions, and the interaction between such vectors can cause them to cancel each other out.

In addition, we can observe that if the tails of these equal-length vectors in the plane are placed together at the same point, the tips of these (planar) vectors all lie on the same circle.

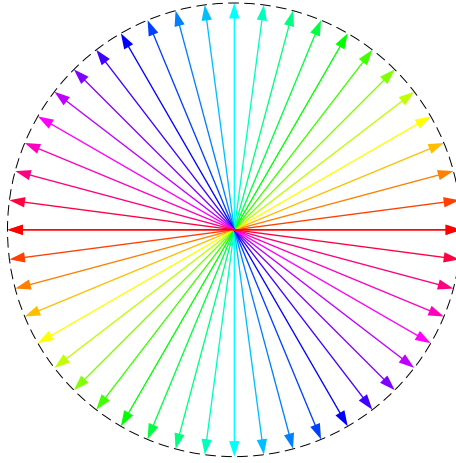


Figure 4. Vectors with Equal Magnitude but Different Directions

The RGB Color Model

In computer graphics, color can also be represented as a vector.

For example, colors in the RGB (Red, Green, Blue) color model can be described using a three-dimensional vector. Each component — R (red), G (green), and B (blue) — represents the intensity of one color.

The dimension of a vector refers to the number of independent components needed to describe it. For example, a two-dimensional vector has two independent components, and a three-dimensional vector has three independent components.

In this book, each of the R, G, and B color components takes a value in the range $[0, 1]$.

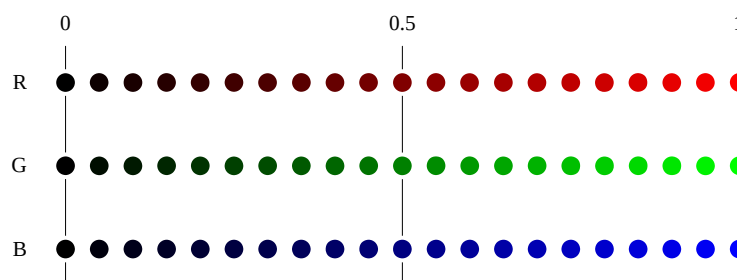


Figure 5. Values of the Three RGB Primary Color Components

When $R = 1$ and G and B are both 0, the resulting color code $(1, 0, 0)$ represents pure red; similarly, the color code for pure green is $(0, 1, 0)$, and for pure blue it is $(0, 0, 1)$. In this book, we use ordinary (half-width) parentheses to denote a tuple. Simply put, a tuple is an ordered sequence of elements. For example, coordinates are often represented as tuples; in the Cartesian plane this can be written as (x_1, x_2) . We use square brackets $[]$ to denote a vector.

In some practical applications, the R, G, and B color components may instead be represented as integers in the range $[0, 255]$, which is another common color encoding scheme.

⚠ It is worth noting that RGB is based on the mixing of light, not the mixing of pigments.

As shown in Figure 6, mixing red light and green light produces yellow light. This is because the superposition of light is additive mixing: when red and green light reach the eye at the same time, what we perceive is yellow.

Similarly, mixing blue light and green light produces cyan, while mixing red light and blue light produces magenta.

When red, green, and blue light are mixed at maximum intensity, the result is white light, because all the colors of the visible spectrum are superimposed together. This principle of additive color mixing is widely used in devices such as monitors, televisions, and projectors, where adjusting the intensities of the red, green, and blue channels can produce a rich variety of colors.

Later in this book, RGB will be used to illustrate a variety of linear algebra concepts.

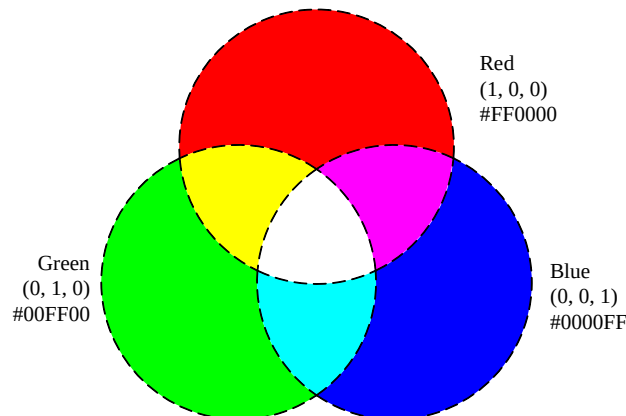


Figure 6. Mixing the Three RGB Primary Colors

Row Vectors

The components of a row vector are arranged in a single row, for example

$$[a_1 \ a_2 \ \cdots \ a_n]$$

The component of the vector at index i is a_i .

The index i ranges over the positive integers from 1 to n .

Again, the dimension of a vector refers to the number of components it contains; clearly, the vector above is an n -dimensional vector.

For example, a two-dimensional row vector has two components, and a three-dimensional row vector has three components.

$$[1 \ 2], \ [1 \ 2 \ 3]$$

In machine learning, the data of a single sample can be represented as a row vector.

Creating a One-Dimensional Array with NumPy

Alongside the linear algebra concepts, this book will show how to perform the corresponding calculations in Python. The book relies heavily on NumPy, a powerful numerical computing library designed for working with multidimensional arrays and matrix operations, which also provides a rich set of linear algebra functions.

The NumPy array is its core data structure. It is more efficient than a Python list and supports vectorized computation, making mathematical operations more concise, faster, and easier to read.

Code 1 first imports the NumPy module and constructs a one-dimensional array from a list containing the numbers 1 through 10. It then uses the array's attributes to obtain its dimension and shape, and shows how to use indexing and slicing to access and extract elements of the array.

As shown in Figure 7, NumPy arrays use zero-based indexing, meaning that indexing starts at 0, so the first element of the array is at index 0. In mathematics, however, we typically start indexing at 1. Therefore, from a mathematical point of view, the first element of a vector has index 1.

It is worth noting that in NumPy, the index of the last component of a one-dimensional array can also be written as -1; the second-to-last component can be indexed as -2, and so on.

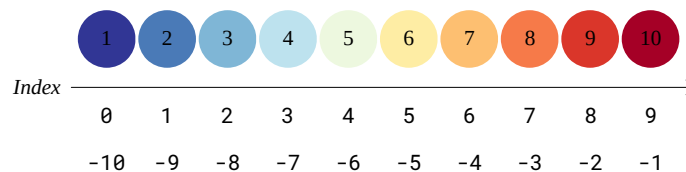


Figure 7. Indexing a One-Dimensional NumPy Array

Code 1. One-Dimensional Array | LA_01_01_01.ipynb

```

## 初始化
a import numpy as np

## 创建数组
b a_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
c a_ = np.array(a_list)
d # a_ = np.arange(1,11)
e dim = a_.ndim
f shape = a_.shape

## 索引
g a_[0]
h a_[1]
i a_-1]
j a_-2]

## 切片
k a_[0:2]
l a_[ :2]
m a_[1:]
n a_-3:]
o a_[7:]

```

Let's now explain Code 1 line by line.

a imports the NumPy module and names it np, so that np can be used later in the code to call the functions and features NumPy provides.

- b** creates a Python list named `a_list`, containing the integers from 1 to 10. A list is a data structure in Python used to store a sequence of data.
- c** uses NumPy's `numpy.array()` function to convert the previously created list `a_list` into a NumPy array, and assigns it to the variable `a_`. A NumPy array is similar to a Python list, but it provides more mathematical operations and functionality.
- d** The `#` at the start of this line marks it as a comment, so it is not executed. The code in the comment shows another way to create the same array: `np.arange(1,11)` generates a sequence of consecutive integers from 1 to 10 (11 not included).
- e** gets the number of “axes” of the array `a_` and stores the result in the variable `dim`. For a one-dimensional array, the value of `ndim` is 1.
- f** gets the shape of the array `a_`, that is, its size along each “axis.” The shape is a tuple; for this one-dimensional array, shape returns `(10,)`, indicating that the array has 10 elements.
- g** takes the element at index 0 of the array. Since indexing starts at 0, this element is the first number, 1.
- h** takes the element at index 1 of the array, that is, the second number, 2.
- i** takes the last element of the array. The negative index -1 refers to the last element, that is, the number 10.
- j** takes the second-to-last element of the array (at index -2).
- k** takes the elements from index 0 to index 2 (index 2 not included). The result is a sub-array containing the first and second elements, namely `array([1, 2])`.
- l** When the start index of a slice is omitted, it defaults to the beginning of the array, so this line also returns `array([1, 2])`.
- m** When the end index of a slice is omitted, it defaults to the end of the array, so this returns all elements from index 1 to the end.
- n** uses a negative index to slice from the end of the array; -3: means taking the elements from the third-to-last element to the end, so it returns `array([8, 9, 10])`.
- o** takes the elements starting from index 7 to the end, returning `array([8, 9, 10])`.

⚠ Note that this book does not specifically cover the basics of Python programming; for that material, please refer to *Python Programming for Beginners* (《编程不难》). This book also does not cover complex visualization techniques for math and data; for that material, please refer to *The Beauty of Visualization* (《可视之美》). The open-source drafts and code for both books are available at <https://github.com/Visualize-ML>.

Column Vectors

The components of a column vector are arranged in a single column.

An n -dimensional column vector $\mathbf{a}$ can be written as

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

Unless stated otherwise, vectors in this book are column vectors by default. In this book, column vectors are denoted by lowercase, bold, italic letters, such as $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, $\mathbf{u}$, $\mathbf{v}$, $\mathbf{x}$, and $\mathbf{y}$.

In summary, row vectors and column vectors are the two basic representations of a vector, and both are widely used in mathematics, physics, and computer science. Their essential difference lies in how the components are arranged, and the choice between a row vector and a column vector usually depends on the specific application and computational needs.

RGB Colors as Column Vectors

As shown in Figure 8, the RGB colors red, green, blue, cyan, magenta, yellow, and white can each be written as a column vector.

The column vector corresponding to white is $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$, and we call this column vector — every component of which is 1 — the all-ones column vector. In this book we denote it as ***1*** (the digit 1 in bold italic).

If there is no light at all, we see black, corresponding to the column vector $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. We call this column vector — every component of which is 0 — the zero vector, denoted as ***0*** (the digit 0 in bold italic).

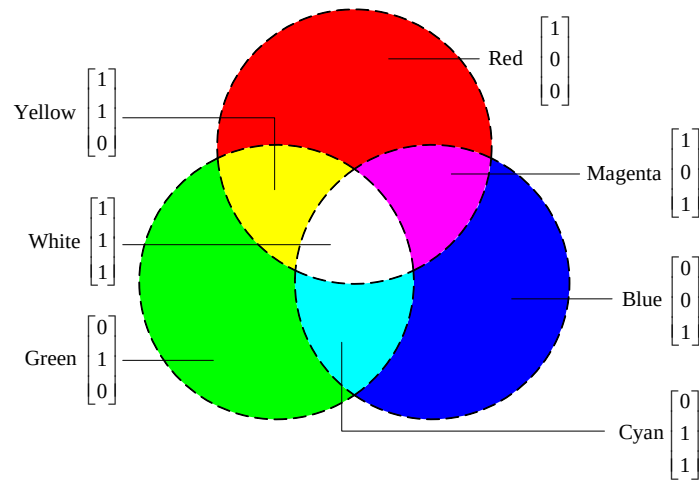


Figure 8. RGB Colors as Column Vectors

You may have already noticed that red + blue = magenta, blue + green = cyan, red + green = yellow, and red + green + blue = white. From the perspective of linear algebra, these operations correspond to vector addition. Later in this chapter, we will introduce the basic operations on vectors.

A Matrix Perspective

To distinguish row vectors from column vectors, this book uses a superscript for row vectors, for example $\mathbf{a}^{(1)}$; and a subscript for column vectors, for example $\mathbf{a}_1$.

For example, the n -dimensional row vector $\mathbf{a}^{(1)}$

$$\mathbf{a}^{(1)} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \end{bmatrix}$$

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

In fact, as shown in Figure 9(a), $\mathbf{a}^{(1)}$ is a matrix with one row and multiple columns.

Simply put, a matrix is a rectangular array arranged in a number of rows and columns, where each component can be uniquely identified by its row index and column index.

⚠ Note that the row index comes first and the column index comes second.

For example, in $\mathbf{a}^{(1)}$ the row index of every component is 1, and the column indices range over the positive integers from 1 to n . The matrix shape corresponding to the row vector shown in Figure 9(a) is denoted $1 \times n$.

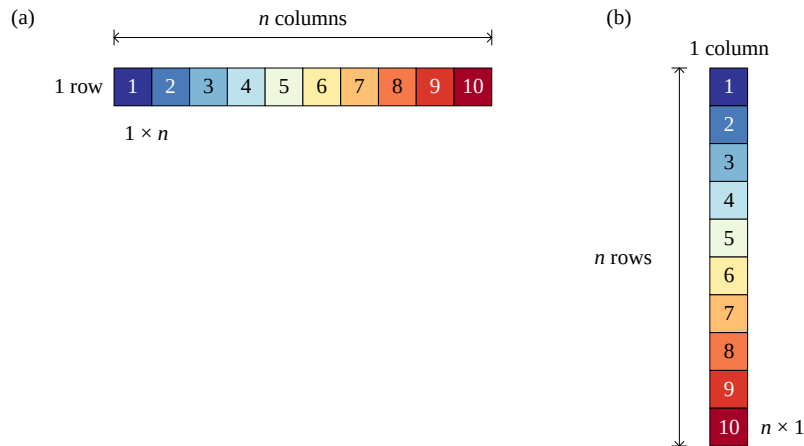


Figure 9. Row and Column Vectors from a Matrix Perspective

As shown in Figure 9(b), the n -dimensional column vector $\mathbf{a}_1$ is

$$\mathbf{a}_1 = \begin{bmatrix} a_{1,1} \\ a_{2,1} \\ \vdots \\ a_{n,1} \end{bmatrix}$$

Similarly, $\mathbf{a}_1$ is a matrix with multiple rows and one column. In $\mathbf{a}_1$, the row indices of the components range over the positive integers from 1 to n , while the column index is always 1.

Figure 9(b) shows that the matrix shape corresponding to the column vector is denoted $n \times 1$.

Transpose swaps the rows and columns of a matrix.

As shown in Figure 10, transposing a row vector turns it into a column vector, and transposing a column vector turns it into a row vector.

In this book, the notation for the transpose of a matrix is a superscript T , as in the example below

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^T = [1 \quad 2 \quad 3], \quad [1 \quad 2 \quad 3]^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Transposition does not change the number of components in a row or column vector, nor does it change the relative positions of the components.

Often, to save vertical space, this book uses a transpose to write a column vector, for example $\mathbf{a} = [3, 4]^T$. This column vector is occasionally also written as $\mathbf{a} = [3; 4]$, where the semicolon ; marks the end of a row.

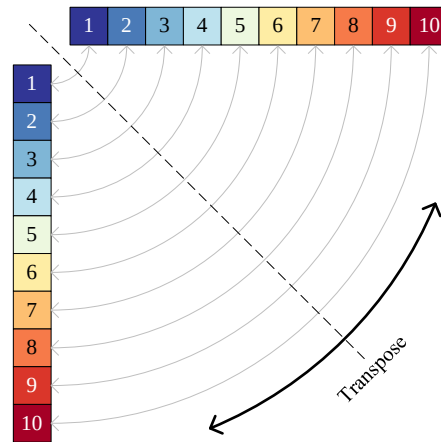


Figure 10. Transposing Between Row Vectors and Column Vectors

Using NumPy to Create a Two-Dimensional Array to Represent a Vector

Code 2 uses NumPy to create a two-dimensional array, constructing a row vector with 1 row and 10 columns, and then obtains a column vector with 10 rows and 1 column through transposition. The example demonstrates indexing and slicing operations on the array, such as accessing specific elements and extracting sub-arrays.

Code 2. One-Dimensional Array |  LA_01_01_02.ipynb

```

## 初始化
import numpy as np

## 创建行向量 (二维)
a row_vector = np.array([[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]])
row_vector.ndim
row_vector.shape

### 索引
b row_vector[0, 0]
row_vector[0, 1]
row_vector[0, -1]

### 切片
c row_vector[:, :2]
row_vector[:, 1:]
row_vector[:, -3:]

## 创建列向量 (二维)

### 转置
d col_vector = row_vector.T

### 索引
e col_vector[0, 0]
col_vector[1, 0]
col_vector[-1, 0]

### 切片
f col_vector[:, :2]
col_vector[1:, :]
col_vector[-3:, :]

```

Let's now explain this code.

a uses the `numpy.array()` function to convert a nested list into a two-dimensional array, where the inner list contains the numbers 1 through 10, and the outer list indicates that this array has only one row — so it forms a 1×10 row vector. `row_vector.ndim` calls the `ndim` attribute of `row_vector` to get the number of dimensions of the array; since the array is stored in two-dimensional form, it returns 2. `row_vector.shape` calls the `shape` attribute of `row_vector`, returning the tuple (1, 10), indicating that the array has 1 row and 10 columns, i.e., its number of rows and columns.

b `row_vector[0, 0]` uses indexing to access the element in row 1, column 1 of the array (note that indexing in Python starts at 0), returning the value 1, which is the first component of the row vector.

`row_vector[0, 1]` accesses the element in row 1, column 2 of `row_vector`, returning the value 2, which corresponds to the second component of the row vector.

`row_vector[0, -1]` uses a negative index to access an element of `row_vector`, where -1 refers to the last column, so it returns the value 10, the last element of the row vector.

c `row_vector[:, :2]` uses a slicing operation, where the colon `:` selects all rows, and `:2` selects the elements at indices 0 and 1 in each row (index 2 not included). The result is a sub-array, `array([[1, 2]])`. Note that this result is still a two-dimensional array.

In `row_vector[:, 1:]`, `1:` means starting from index 1 through to the end, and the colon selects all rows.

`row_vector[:, -3:]` uses slicing to select, in each row, the elements from the third-to-last to the last; `-3:` means starting from the third-to-last column.

d uses the `.T` attribute to transpose the row vector `row_vector`, producing a column vector — that is, transforming the original array with one row and multiple columns into an array with multiple rows and one column.

e `col_vector[0, 0]` uses indexing to access the element in row 1, column 1 of the transposed column vector, returning the value 1, the first component of the column vector.

`col_vector[1, 0]` accesses the element in row 2, column 1 of `col_vector`, returning the value 2, which corresponds to the second component of the column vector.

`col_vector[-1, 0]` uses the negative index `-1` to refer to the last row, accessing the last element of the transposed column vector, returning the value 10, the last component of the column vector.

f `col_vector[:2, :]` uses a slicing operation, where `:2` selects the first two rows and the colon `:` selects all columns, so it returns a sub-array containing the first two elements of the column vector. This sub-array is also a two-dimensional array.

`col_vector[1:, :]` uses a slicing operation, where `1:` means starting from row 2 through to the last row, selecting all columns, so it returns the sub-array formed by the second element through the last element.

`col_vector[-3:, :]` In the slicing operation of this line, `-3:` means selecting from the third-to-last row through to the last row, and the colon means all columns, so it returns the sub-array containing the last three elements of the column vector.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Please install Anaconda and learn to use JupyterLab; you can refer to the first three chapters of *Programming Is Not Hard*. The open-source draft is available at:

https://github.com/Visualize-ML/Book1_Python-For-Beginners

Q2. Please study the concepts of list, tuple, and variable in Python.

Q3. Define the one-dimensional NumPy array `numpy.array([1, 2, 3, 4, 5, 6])`, and extract from it

► the odd numbers;

► the even numbers.

Q4. Define the two-dimensional NumPy array `numpy.array([[ -3, -2, -1, 0, 1, 2, 3]])`, and extract from it

► the components greater than 0;

► all components greater than or equal to 0;

► all components less than 2;

► all components less than or equal to 2.

Q5. Perform a transpose operation on the one-dimensional array `numpy.array([1, 2, 3, 4, 5, 6])`, and explain the result.

Q6. Perform a transpose operation on the two-dimensional array `numpy.array([[ -3, -2, -1, 0, 1, 2, 3]])`, and explain the result.

Q7. Please study and try out each of the following NumPy functions.

► `numpy.linspace()`

► `numpy.zeros()`

► `numpy.ones()`

► `numpy.random.rand()`

Q8. Please learn how to define colors in Matplotlib.

<https://matplotlib.org/stable/users/explain/colors/colors.html>

Q9. Ask DeepSeek/ChatGPT to comment the following example line by line, and learn how to plot a streamplot.

https://matplotlib.org/stable/gallery/images_contours_and_fields/plot_streamplot.html

Q10. Ask DeepSeek/ChatGPT to comment the following example line by line, and learn how to plot a 3D quiver (arrow) diagram.

<https://matplotlib.org/stable/gallery/mplot3d/quiver3d.html>

Q11. Please study the concepts of list, tuple, and variable in Python.